Python Engineering

WEEK 03 - Modules & Coordinates | CLASS 10 - Advanced Functions | EX18

Goal: learn positional, keyword, and default arguments, *args, **kwargs, unpacking, and functions passed as arguments.

Reference values for EX18

Use these values so every block has a clear numerical reference. To demonstrate gravity override we use h0 = 100 m: if h0 were 0 m, the g*h0 term would be zero and changing g would not change initial energy.

Variable / case	Reference value	Purpose
h0	100.0 m	Makes the effect of changing g visible
v0	50.0 m/s	Initial velocity
default g	9.81 m/s^2	Default value in initial_energy()
override g	1.62 m/s^2	Example overriding the default
measurements	[10.2, 10.5, 9.9, 10.1]	Unpacking with *
summary	129.9 m, 10.3 s, -51.0 m/s	**kwargs
x, y	5, 3	Function as argument

Main expected results

```
initial_energy(100.0, 50.0)
# g = 9.81 -> 2231.0 J/kg

initial_energy(100.0, 50.0, g=1.62)
# override g -> 1412.0 J/kg

average_measurements(*[10.2, 10.5, 9.9, 10.1])
# 10.175

apply_operation(add, 5, 3)
# 8

apply_operation(multiply, 5, 3)
# 15
```

BLOCK A - Positional, keyword, and default arguments

```
def initial_energy(h0, v0, g=9.81):
    return g * h0 + 0.5 * v0**2

initial_energy(100.0, 50.0)
initial_energy(h0=100.0, v0=50.0)
initial_energy(100.0, 50.0, g=1.62)
```

Compare positional calling, keyword calling, and overriding the default value.

BLOCK B - *args

```
def average_measurements(*values):
    return sum(values) / len(values)

measurements = [10.2, 10.5, 9.9, 10.1]
average_measurements(*measurements)
```

BLOCK C - **kwargs

```
def simulation_summary(**kwargs):
    for key, value in kwargs.items():
```

```
        print(key, value)

simulation_summary(
    max_altitude=129.9,
    flight_time=10.3,
    final_velocity=-51.0
)
```

BLOCK D - Dictionary unpacking with **

```
config = {'h0':100.0, 'v0':50.0, 'g':9.81}
initial_energy(**config)
```

BLOCK E - Function as an argument

```
def add(a, b):
    return a + b

def multiply(a, b):
    return a * b

def apply_operation(operation, x, y):
    return operation(x, y)

apply_operation(add, 5, 3)
apply_operation(multiply, 5, 3)
```

FINAL VALIDATION

```
Explain in comments:
- Difference between positional and keyword arguments
- What a default argument does
- What *args receives and its type
- What **kwargs receives and its type
- Difference between * when defining and * when calling
- Difference between ** when defining and ** when calling
- Why apply_operation(add, 5, 3) passes add without ()
```

Completion criterion

EX18 is complete when you can intentionally choose the appropriate argument style and can pass one function to another while understanding that a function can also be treated as an object.